

FreeSSM reconnect izmene - tehnicki report

Datum: 2026-06-08

Ovaj report opisuje konkretne izmene koje su napravljene da Control Unit prozor ostane otvoren kada se izgubi OBD/SSM komunikacija, i da korisnik moze ponovo da klikne Connect kada se veza vrati.

Kratak pregled logike

- Communication error vise ne zatvara Control Unit dijalog.
- Stara SSM komunikacija se resetuje i brise, a dijagnosticki interfejs se disconnect-uje.
- UI se vraća u disconnected stanje: Connect dugme je aktivno, a funkcijska dugmad su neaktivna dok nema veze.
- Pamti se aktivni mod, pa Connect pokusava da ponovo otvori isti ekran: DCs, MBs/SWs, Adjustments ili System Tests.
- Kod Measuring Blocks moda cuva se izbor senzora preko postojećeg _lastMBSWmetaList mehanizma.

1. Dodata interna stanja za reconnect zastitu

Lokacija: src/ControlUnitDialog.cpp, oko linije 39

Status izmene: Izmenjen postojeci konstruktor; dodate su dve nove inicijalizacije.

1	_setup_done = false;
2	_connected = false;
3	_handlingCommunicationLoss = false;
4	_communicationErrorActive = false;
5	_mode = Mode::None;
6	_startupContentSelection = ContentSelection::DCsMode;
7	_startupAutostart = false;

_handlingCommunicationLoss sprečava rekursivno ciscenje ako error signal stigne dok se stari widgeti brisu.
_communicationErrorActive se koristi u closeEvent-u da dialog ne prihvati zatvaranje dok korisnik zatvara error poruku.

2. Deklaracije u ControlUnitDialog.h

Lokacija: src/ControlUnitDialog.h, oko linije 98

Status izmene: Izmenjen header; dodata su polja, pomocna metoda i slot.

1	bool _setup_done;
2	bool _connected;
3	bool _handlingCommunicationLoss;
4	bool _communicationErrorActive;
5	Mode _mode;
6	ContentSelection _startupContentSelection;
7	QString _startupMBSWselectionFile;
8	bool _startupAutostart;
9	// Content backup parameters:
10	std::vector<MBSWmetadata_dt> _lastMBSWmetaList;
11	MBSWsettings_dt _MBSWsettings;
12	
13	virtual QString systemName() = 0;
14	virtual QString controlUnitName() = 0;
15	virtual CUnitType controlUnitType() = 0;
16	virtual bool systemRequiresManualON() = 0;

```

17     virtual CUcontent_DCs_abstract * allocate_DCsContentWidget() = 0;
18     bool displaySystemDescriptionAndID(SSMprotocol *SSMPdev, CUinfo_abstract *abstractInfowidget);
19     virtual bool displayExtendedCUinfo(SSMprotocol *SSMPdev, CUinfo_abstract *abstractInfowidget, FSSM_ProgressDialog *initstatusmsgbox = NULL) = 0;
20     bool prepareContentWidget(Mode mode);
21     void setContentWidget(QString title, QWidget *contentwidget);
22     void deleteContentWidgets();
23     QWidget * contentWidget();
24     bool getParametersFromCmdLine(QStringList *cmdline_args, QString *selection_file, bool *autostart);
25     bool getModeForContentSelection(ContentSelection csel, Mode *mode);
26     bool initializeControlUnit(Mode mode, bool showSuccessMessage = true);
27     void applyMBSWStartupParameters(ContentSelection csel);
28     void handleCommunicationLoss();
29     void setDisconnectedState(QString message);
30     void setContentSelectionButtonEnabled(ContentSelection csel, bool enabled);
31     void setContentSelectionButtonChecked(ContentSelection csel, bool checked);
32     SSMprotocol::CUsetupResult_dt probeProtocol(CUtype CUtype);
33     bool handleActuatorTests(FSSM_ProgressDialog *statusmsgbox);
34     bool startMode(Mode mode);
35     bool startDCsMode();
36     bool startMBSWsMode();
37     bool startAdjustmentsMode();
38     bool startSystemOperationTestsMode();
39     void runClearMemory(SSMprotocol::CMlevel_dt level);
40     void saveContentSettings();
41     void closeEvent(QCloseEvent *event);
42
43 private slots:
44     void connectToControlUnit();
45     void switchToDCsMode();
46     void switchToMBSWsMode();
47     void switchToAdjustmentsMode();
48     void switchToSystemOperationTestsMode();
49     void clearMemory();
50     void clearMemory2();
51     void connectionLost();
52     void communicationError(QString addstr = "");

```

Header sada zna za centralnu metodu handleCommunicationLoss() i slot connectionLost(). Time svi izvori gresaka mogu da udju u isti reconnect tok umesto da direktno zatvaraju prozor.

3. setDisconnectedState() - Connect se vraća, funkcije se gasi

Lokacija: src/ControlUnitDialog.cpp, oko linije 466

Status izmene: Izmenjena postojeća funkcija; dodato deaktiviranje content selection dugmadi.

```

1     void ControlUnitDialog::setDisconnectedState(QString message)
2     {
3         _connected = false;
4         if (_connect_pushButton)
5         {
6             _connect_pushButton->setEnabled(true);
7             _connect_pushButton->setText(tr("Connect"));
8         }
9         QList<ContentSelection> selections = _contentSelectionButtons.keys();
10        for (int k=0; k<selections.size(); k++)
11            setContentSelectionButtonEnabled(selections.at(k), false);
12    #ifndef SMALL_RESOLUTION
13        if (_ifstatusbar)
14        {
15            _ifstatusbar->setProtocolName(message, Qt::darkYellow);
16            _ifstatusbar->setBaudRate("---");

```

```

17     }
18     #endif
19 }

```

Ova funkcija postavlja stanje bez konekcije. Connect dugme se ponovo ukljucuje i dobija tekst Connect. Dodata petlja gasi ostalu funkcionalnu navigaciju dok ECU nije ponovo inicijalizovan.

4. connectToControlUnit() - reconnect ide na aktivni mod

Lokacija: src/ControlUnitDialog.cpp, oko linije 800

Status izmene: Izmenjena postojeća funkcija.

```

1  void ControlUnitDialog::connectToControlUnit()
2  {
3      Mode mode;
4      if (_connected)
5          return;
6      mode = _mode;
7      if ((mode == Mode::None) && !getModeForContentSelection(_startupContentSelection, &mode))
8          mode = Mode::DCs;
9      if (initializeControlUnit(mode, true))
10         return;
11     QMessageBox msg(QMessageBox::Information, tr("Not connected"),
12                     tr("The Control Unit did not answer yet.\nSwitch ignition on and try again."),
13                     QMessageBox::Ok, this);
14     QFont msgfont = msg.font();
15     msgfont.setPointSize(9);
16     msg.setFont(msgfont);
17     msg.show();
18     msg.exec();
19     msg.close();
20 }

```

Ranije se Connect oslanjao prvenstveno na startup selection. Sada prvo uzima _mode, odnosno ekran koji je bio aktivan pre gubitka konekcije. Ako _mode nije poznat, koristi se stari fallback na startup izbor ili DCs.

5. Error signali vise ne zatvaraju dijalog

Lokacija: src/ControlUnitDialog.cpp, oko linije 944

Status izmene: Izmenjene postojeće funkcije startDCsMode, startMBsSWsMode, startAdjustmentsMode i startSystemOperationTestsMode.

```

1  bool ControlUnitDialog::startDCsMode()
2  {
3      int DCgroups = 0;
4      if (_content_DCs == NULL)
5          return false;
6      if (!_content_DCs->setup(_SSMPdev))
7          return false;
8      if (!_SSMPdev->getSupportedDCgroups(&DCgroups))
9          return false;
10     if (DCgroups != SSMprotocol::noDCs_DCgroup)
11     {
12         if (!_content_DCs->startDCreading())
13             return false;
14         connect(_content_DCs, SIGNAL( error() ), this, SLOT( connectionLost() ) );
15     }
16     _mode = Mode::DCs;
17     return true;

```

```

18     }
19
20     bool ControlUnitDialog::startMBsSwsMode()
21     {
22         if (_content_MBsSws == NULL)
23             return false;
24         if (!_content_MBsSws->setup(_SSMPdev))
25             return false;
26         if (!_lastMBSWmetaList.empty())
27         {
28             if (!_content_MBsSws->setMBSWselection(_lastMBSWmetaList))
29                 return false;
30         }
31         connect(_content_MBsSws, SIGNAL( error() ), this, SLOT( connectionLost() ) );
32         _mode = Mode::MBsSws;
33         return true;
34     }
35
36     bool ControlUnitDialog::startAdjustmentsMode()
37     {
38         if (_content_Adjustments == NULL)
39             return false;
40         if (!_content_Adjustments->setup(_SSMPdev))
41             return false;
42         connect(_content_Adjustments, SIGNAL( communicationError() ), this, SLOT( connectionLost() ) );
43         _mode = Mode::Adjustments;
44         return true;
45     }
46
47     bool ControlUnitDialog::startSystemOperationTestsMode()
48     {
49         if (_content_SysTests == NULL)
50             return false;
51         if (!_content_SysTests->setup(_SSMPdev))
52             return false;
53         connect(_content_SysTests, SIGNAL( error() ), this, SLOT( connectionLost() ) );
54         _mode = Mode::SysTests;
55         return true;
56     }

```

Najbitnija promena je zamena SLOT(close()) sa SLOT(connectionLost()). Kada unutrašnji widget prijavi gresku, Control Unit prozor ostaje otvoren i prelazi u reconnect stanje.

6. runClearMemory() - abort/reconnect failure vise ne zatvara prozor

Lokacija: src/ControlUnitDialog.cpp, oko linije 1032

Status izmene: Izmenjena postojeća funkcija.

```

1     void ControlUnitDialog::runClearMemory(SSMprotocol::CMlevel_dt level)
2     {
3         bool ok = false;
4         ClearMemoryDlg::CMresult_dt result;
5         // Create "Clear Memory"-dialog:
6         ClearMemoryDlg cmdlg(this, _SSMPdev, level);
7         // Temporary disconnect from "communication error"-signal:
8         disconnect(_SSMPdev, SIGNAL( commError() ), this, SLOT( communicationError() ));
9         // Run "Clear Memory"-procedure:
10        result = cmdlg.run();
11        // Reconnect to "communication error"-signal:
12        connect(_SSMPdev, SIGNAL( commError() ), this, SLOT( communicationError() ));
13        // Check result:
14        if ((result == ClearMemoryDlg::CMresult_success) && (_mode == Mode::Adjustments))

```

15	{
16	FSSM_WaitMsgBox waitmsgbox(this, tr("Reading Adjustment Values... Please wait !"));
17	waitmsgbox.show();
18	ok = _content_Adjustments->setup(_SSMPdev); // refresh adjustment values
19	waitmsgbox.close();
20	if (!ok)
21	communicationError();
22	}
23	else if (result == ClearMemoryDlg::CMresult_communicationError)
24	{
25	communicationError();
26	}
27	else if ((result == ClearMemoryDlg::CMresult_reconnectAborted) (result == ClearMemoryDlg::CMresult
28	_reconnectFailed))
29	{
30	handleCommunicationLoss();
31	}

Ako Clear Memory procedura zavrsi sa reconnectAborted ili reconnectFailed, ranije se pozivao close(). Sada se poziva handleCommunicationLoss(), pa korisnik ostaje u Control Unit dijalogu i moze da pokusa Connect.

7. handleCommunicationLoss() - nova centralna funkcija

Lokacija: src/ControlUnitDialog.cpp, oko linije 1047

Status izmene: Cela funkcija je dodata.

```
1 void ControlUnitDialog::handleCommunicationLoss()
2 {
3     Mode mode = _mode;
4
5     if (_handlingCommunicationLoss)
6         return;
7     _handlingCommunicationLoss = true;
8     saveContentSettings();
9     if (_SSMPdev)
10    {
11        disconnect( _SSMPdev, SIGNAL( commError() ), this, SLOT( communicationError() ) );
12        _SSMPdev->resetCUdata();
13    }
14    deleteContentWidgets();
15    if (_SSMPdev)
16    {
17        delete _SSMPdev;
18        _SSMPdev = NULL;
19    }
20    if (_diagInterface->isConnected())
21        _diagInterface->disconnect();
22    _connected = false;
23    setDisconnectedState(tr("Connection lost. Switch ignition on, check the diagnostic interface, then press Connect."));
24    if (mode != Mode::None)
25    {
26        prepareContentWidget(mode);
27        _mode = mode;
28    }
29    _handlingCommunicationLoss = false;
30 }
```

Ovo je srce nove logike. Funkcija pamti trenutni mod, cuva MB/SW izbor, resetuje SSM CU podatke, brise stare widgete vezane za staru konekciju, brise stari SSMprotocol objekat, disconnect-uje interfejs, zatim vraća UI u disconnected stanje i ponovo priprema isti tip content widgeta za sledeći Connect.

8. connectionLost() - novi slot za interne greske

Lokacija: src/ControlUnitDialog.cpp, oko linije 1079

Status izmene: Cela funkcija je dodata.

```
1 void ControlUnitDialog::connectionLost()
2 {
3     handleCommunicationLoss();
4 }
```

Ovaj slot je tanak wrapper: svi error signali iz unutrašnjih widgeta ulaze kroz njega, a on poziva centralni reconnect reset.

9. communicationError() - OK vise ne zatvara Control Unit prozor

Lokacija: src/ControlUnitDialog.cpp, oko linije 1085

Status izmene: Izmenjena postojeća funkcija; uklonjena je close() logika i dodata reconnect obrada.

```

1 void ControlUnitDialog::communicationError(QString addstr)
2 {
3     // Show error message
4     if (addstr.size() > 0) addstr.prepend('\n');
5     _communicationErrorActive = true;
6     QMessageBox msg( QMessageBox::Critical, tr("Communication Error"), tr("Communication Error:\n- No or
invalid answer from Control Unit -") + addstr, QMessageBox::Ok, this);
7     QFont msgfont = msg.font();
8     msgfont.setPointSize(9);
9     msg.setFont( msgfont );
10    msg.show();
11    msg.exec();
12    msg.close();
13    handleCommunicationLoss();
14    _communicationErrorActive = false;
15 }

```

Poruka o gresci ostaje ista za korisnika. Razlika je posle OK: umesto close(), poziva se handleCommunicationLoss(). Flag _communicationErrorActive se koristi da closeEvent ignorise pokusaj zatvaranja koji bi mogao da stigne dok je error tok aktivan.

10. closeEvent() - zastita od zatvaranja tokom error toka

Lokacija: src/ControlUnitDialog.cpp, oko linije 1102

Status izmene: Izmenjena postojeća funkcija.

```

1 void ControlUnitDialog::closeEvent(QCloseEvent *event)
2 {
3     if (_communicationErrorActive)
4     {
5         event->ignore();
6         return;
7     }
8     if (_SSMPdev)
9     {
10        // Create wait message box:
11        FSSM_WaitMsgBox waitmsgbox(this, tr("Stopping Communication... Please wait !"));
12        waitmsgbox.show();
13        // Reset CU data:
14        _SSMPdev->resetCUdata();
15        // Close wait message box:
16        waitmsgbox.close();
17    }
18    event->accept();
19 }

```

Ako se closeEvent desi dok je aktivna obrada communication error-a, event se ignorise. Normalno korisnicko zatvaranje i dalje radi kada _communicationErrorActive nije aktivan.

11. ActuatorTestDlg.cpp - uklonjen direktni parent->close()

Lokacija: src/ActuatorTestDlg.cpp, oko linije 85

Status izmene: Izmenjena postojeća grana u konstruktoru ActuatorTestDlg.

```

1     QMessageBox errmsg( QMessageBox::Critical, tr("Communication Error"), tr("Please ensure that
ignition is switched ON."), QMessageBox::NoButton, this);
2     errmsg.addButton(tr("Retry"), QMessageBox::AcceptRole);
3     errmsg.addButton(tr("Exit Control Unit"), QMessageBox::RejectRole);

```

4	msgfont = errmsg.font();
5	msgfont.setPointSize(9);
6	errmsg.setFont(msgfont);
7	errmsg.show();
8	dlgchoice = errmsg.exec();
9	errmsg.close();
10	// close dialog if aborted
11	if (dlgchoice == 1)
12	{
13	close(); // close actuator test dialog
14	emit error();
15	return;

Ovaj deo je ranije mogao direktno da zatvori roditeljski Control Unit prozor preko parent->close(). Sada zatvara samo Actuator Test dijalog i emituje error signal, koji ControlUnitDialog prevodi u connectionLost tok.

Kako sada treba da radi u aplikaciji

1. Korisnik otvori Control Unit dijalog i poveze se na ECU.
2. Ako se izgubi OBD/SSM konekcija, prikazuje se Communication Error poruka.
3. Korisnik klikne OK.
4. Control Unit prozor ostaje otvoren.
5. Stari SSM objekat i stari content widget se uklanjaju, a interfejs se disconnect-uje.
6. Status bar dobija poruku da je veza izgubljena, a Connect dugme je ponovo aktivno.
7. Kada se fizicka veza/ignition/interfejs vrati, korisnik klikne Connect.
8. Program ponovo inicijalizuje isti Control Unit i pokusava da vrati isti aktivni mod.
9. Ako je korisnik bio u Measuring Blocks modu, izbor senzora se cuva u memoriji i pokusava da se vrati posle reconnect-a.

Linux build i pokretanje

Projekat je qmake Qt aplikacija. README navodi da su podrzani Qt4, Qt5 i eksperimentalno Qt6. Na modernom Linuxu najprakticnije je probati Qt5 build.

Primer za Debian/Ubuntu:

```
1 sudo apt update
2 sudo apt install build-essential qtbase5-dev qtools5-dev-tools libqt5support5-dev
3
4 cd /putanja/do/FreeSSM_profesor_nova_verzija
5 qmake-qt5
6 make release
7 make translation
8 make release-install
9
10 cd ~/FreeSSM
11 ./FreeSSM
```

Ako sistem koristi drugo ime za qmake:

```
1 qmake
2 # ili:
3 qmake6
4
5 make release
```

Mala rezolucija:

```
1 qmake-qt5 "CONFIG+=small-resolution"
2 make release
```

Custom install folder:

```
1 qmake-qt5 "INSTALLDIR=/home/$USER/FreeSSM_TEST"
2 make release
3 make release-install
4
5 cd /home/$USER/FreeSSM_TEST
6 ./FreeSSM
```

Napomena: Ako se koristi J2534 interfejs, na Linuxu se instalira i definitions/J2534libs.xml kroz unix install target. Za obican serial/USB interfejs najbitnije je da korisnik ima pristup serijskom portu, npr. clan grad grupe dialout na Debian/Ubuntu sistemima.

```
1 sudo usermod -aG dialout $USER
2 # zatim logout/login
```